// Tema 3
// Estructures
// Dinàmiques

TEMA 3 - ESTRUCTURES DINÀMIQUES

Llistes i Iteradors

Estructures enlazadas que permiten inserción y borrado O(1) en cualquier posición

Ventaja clave:

Las listas permiten inserción y borrado O(1) en cualquier posición, a diferencia de los vectores que requieren O(n) porque hay que mover elementos en memoria.

// Nodos enlazados
// Iteradores
// O(1) inserció

TEMA 3 - MOTIVACIÓ

El Problema dels Vectors

Inserción y borrado requieren mover todos los elementos posteriores

Insertar 15 en posición 2

ESTADO INICIAL

10	20	30	40

PASO 1: DESPLAZAR ELEMENTOS

10	20	30	40	?

PASO 2: INSERTAR ELEMENTO

10	20	15	30	40

Coste de operación:

Hay que mover 2 elementos (30 y 40) → O(n) en general

Análisis de Complejidad

Operación	Vector
Acceso [i]	O(1)
Insertar	O(n) ❌
Borrar	O(n) ❌
Recorrer	1x (rápido)

Solución: Usar Listas

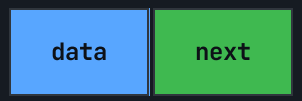
Las listas permiten inserción y borrado O(1) en cualquier posición, pero el recorrer es más lento.

TEMA 3 - ESTRUCTURA

Estructura de Nodos Enlazados

Cómo se organizan los datos en memoria mediante punteros

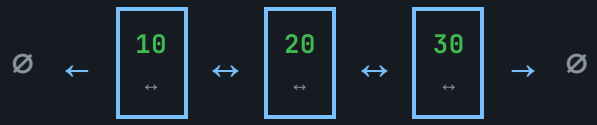
Nodo Simple (next)



Cada nodo contiene: un valor (data) y un puntero al siguiente nodo (next). El último nodo apunta a nullptr.

Nodo Doblemente Enlazado (prev + next)

Cada nodo contiene: puntero anterior (prev), valor (data) y puntero siguiente (next). Permite recorridos en ambas direcciones.



TEMA 3 - COMPARATIVA

Vector vs Lista: Trade-offs

Análisis de complejidad y características de ambas estructuras

vector<T>

Acceso [i]	O(1) ✓
Insertar	O(n) ✗
Borrar	O(n) ✗
Recorrer	1x (rápido)
Memoria	Continua

Ventajas

- ✓ Acceso aleatorio O(1)
- ✓ Recorrer muy rápido
- ✓ Cache-friendly

Desventajas

- ✓ Inserción O(n)
- ✓ Borrado O(n)
- ✓ Requiere realocación

list<T>

Acceso [i]	O(n) ✗
Insertar	O(1) ✓
Borrar	O(1) ✓
Recorrer	100-200x lent
Memoria	Dispersa

Ventajas

- ✓ Inserción O(1)
- ✓ Borrado O(1)
- ✓ Sin realocación

Desventajas

- ✓ Acceso O(n)
- ✓ Recorrer lento
- ✓ No cache-friendly

TEMA 3 - REFERENCIA

Métodos de list<T>

Operaciones principales para manipular listas

+ Inserción y Acceso

Método	Descripción
push_back(x)	Afegir x al final
push_front(x)	Afegir x al principi
insert(it, x)	Inserir x a posició it
front()	Referència al primer
back()	Referència al últim

Borrado

Método	Descripción
pop_back()	Eliminar últim
pop_front()	Eliminar primer
erase(it)	Eliminar a posició it
empty()	¿Está buida?
size()	Nombre d'elements

Ejemplo: Inserción

```
list<int> L = {10, 20, 30};
L.push_back(40); // L = {10,20,30,40}
L.push_front(5); // L = {5,10,20,30,40}
auto it = L.begin();
++it; // apunta a 10
L.insert(it, 7); // L = {5,7,10,20,30,40}
```

Ejemplo: Borrado

```
list<int> L = {10, 20, 30, 40};
L.pop_back(); // L = {10,20,30}
L.pop_front(); // L = {20,30}
auto it = L.begin();
++it; // apunta a 30
L.erase(it); // L = {20}
```

TEMA 3 - ITERADORES

Iteradores: Concepto Fundamental

Marcadores de posición para recorrer y modificar contenedores

¿Qué es un Iterador?

Un **iterador** es un objeto que representa una **posición** en un contenedor. Es como una **flecha** que marca dónde estamos en la lista, permitiendo acceder y modificar el elemento en esa posición.

Visualización

10	20	30	40

↑ ITERADOR APUNTA AQUÍ

Clave: Las listas **no tienen índices** como los vectores. Los iteradores son la **única forma** de acceder a los elementos.

Tipos de Iteradores

iterator

Iterador normal. Permite leer y modificar elementos. Recorre hacia adelante.

const_iterator

Iterador constante. Solo lectura. No se puede modificar el elemento.

reverse_iterator

Iterador inverso. Recorre hacia atrás (desde final a principio).

const_reverse_iterator

Iterador inverso constante. Solo lectura, hacia atrás.

Operaciones Básicas

Navegación y Acceso

- `++` Avanzar al siguiente elemento
- `--` Retroceder al anterior
- `*` Acceder al elemento actual
- `==` Comparar posiciones
- `≠` Verificar si son distintos

TEMA 3 - ITERADORES

Operaciones con Iteradores

Cómo manipular y navegar iteradores en listas

Operaciones Básicas

Operación	Descripción
`++it`	Avanzar al siguiente elemento
`--it`	Retroceder al anterior
`*it`	Acceder al elemento
`it == it2`	Comparar posiciones
`it != it2`	Diferente posición
`begin()`	Primer elemento
`end()`	Sentinella (pasado final)

Ejemplo Práctico

Código:

```
list<int> L = {10, 20, 30};
auto it = L.begin();
it++; // 2° elemento
*it += 5; // 20 → 25
```

Estado 1: begin()

Estado 2: it++

Estado 3: *it += 5

TEMA 3 - EJERCICIO 1

Inserción Ordenada (sorted_insert)

Algoritmo paso a paso: buscar posición y insertar con coste O(1)

Código del Algoritmo

```
void sorted_insert(
  list<int>& L, int x) {

  list<int>::iterator it
    = L.begin();

  // Buscar posición
  while (it ≠ L.end()
        and *it < x) {
    ++it;
  }

  // Insertar O(1)
  L.insert(it, x);
}
```

Clave:

insert() retorna un iterador al elemento insertado. El coste es O(1) porque solo hay que cambiar punteros.

Ejecución: sorted_insert(L, 15)

PASO 1: ESTADO INICIAL

10	20	30

it = begin() (apunta a 10)

PASO 2: AVANZAR IT

10	20	30

*it = 20 < 15? NO → salir bucle

PASO 3: INSERTAR EN POSICIÓN IT

10	15	20	30

L.insert(it, 15) → O(1)

RESULTADO FINAL

10	15	20	30

Lista ordenada correctamente

TEMA 3 - EJERCICIO 2

Insertar Ceros en Bucle

Problema: invalidación de iteradores y solución con advance()

⚠ Problema

```
// INCORRECTO
for (auto it = L.begin(); it ≠ L.end(); it++) {
  if (*it ≥ '1' && *it ≤ '9') {
    it = L.insert(it, '0');
    // it apunta al '0' y se salta el dígito
  }
}
```

⚠ El Problema:

insert() devuelve el iterador al elemento insertado; it++ salta el dígito original.

✔ Solución

```
// CORRECTO
for (auto it = L.begin(); it ≠ L.end(); it++) {
  if (*it ≥ '1' && *it ≤ '9') {
    it = L.insert(it, '0');
    advance(it, 2);
  }
}
```

✓ La Solución:

advance(it, 2) salta el '0' insertado y el dígito original, permitiendo que la siguiente iteración proceda.

▶ Ejecución: Insertar '0' antes de dígitos

ESTADO INICIAL

3	a	5	b

it = begin() (apunta a '3')

PASO 1: INSERTAR '0' ANTES DE '3'

0	3	a	5	b

it = L.insert(it, '0') → apunta a '0'

PASO 2: ADVANCE(IT, 2)

0	3	a	5	b

it avanza 2: salta '0' y '3'

PASO 3: IT++ (SIGUIENTE ITERACIÓN)

0	3	a	5	b

it apunta a '5' (correcto)

RESULTADO FINAL

0	3	a	0	5	b

Todos los dígitos tienen '0' antes

TEMA 3 - EJERCICIO 3

Eliminar Elementos en Bucle

Técnica correcta: usar el iterador retornado por erase()

✕ Código Incorrecto

```
void erase_negative(list<double>& L) { for (auto it = L.begin(); it ≠ L.end();
++it) { if (*it < 0) { L.erase(it); // it inválido } } }
```

Problema:

Después de erase(), it queda inválido. ++it sobre él → comportamiento indefinido.

✓ Código Correcto

```
void erase_negative(list<double>& L) {
  auto it = L.begin();
  while (it ≠ L.end()) {
     if (*it < 0) it = L.erase(it); /* it apunta al siguiente */
     else ++it;
  }
}
```

▶ Ejecución: erase_negative(L)

ESTADO INICIAL

10 | -5 | 20 | -3 | 30

Rojo: elementos a eliminar

PASO 1: IT APUNTA A 10

↓ it 10 | -5 | 20 | -3 | 30

10 ≥ 0 → ++it

PASO 2: IT APUNTA A -5

10 | ↓ it -5 | 20 | -3 | 30

-5 < 0 → erase(it)

PASO 3: ERASE() RETORNA SIGUIENTE

10 | ↓ it 20 | -3 | 30

it = erase(it) → apunta a 20

Clave:

erase() retorna un iterador al siguiente elemento válido; asignarlo a it permite continuar correctamente.

TEMA 3 - ITERADORES INVERSOS

Recorridos Descendentes

Iteradores inversos: rbegin() y rend() para recorrer de final a principi

TEMA 3 - CIERRE

Resumen: Llistes i Iteradors

Conceptos clave y aplicaciones en sistemas reales

Conceptos Clave

Estructura Enlazada
Nodos con punteros (next, prev) permiten flexibilidad en memoria

Inserción O(1)
A diferencia de vectores, insertar/borrar es O(1) en cualquier posición

Iteradores
Marcadores de posición que reemplazan índices. Operaciones: ++, --, *

Recorrido Secuencial
No hay acceso aleatorio. Recorrer es 100-200x más lento que vectores

Trade-offs
Elegir entre acceso rápido (vector) o inserción rápida (lista)

Aplicaciones Reales

Undo/Redo
Guardar acciones en lista. Deshacer = ir atrás, rehacer = ir adelante

Cola de Impresión
Documentos se insertan/eliminan frecuentemente. Inserción O(1) es crítica

Playlist de Música
Agregar/quitar canciones frecuentemente. Recorrido secuencial es suficiente

Historial Navegador
Atrás/Adelante requieren iteradores. Inserción O(1) para nuevas páginas

Gestión de Procesos
Procesos se crean/destruyen dinámicamente. Listas permiten flexibilidad

Próximo Tema:
Tema 4 - Arbres Binaris de Búsqueda (BST)